Bachelor Thesis

Roman Sannikov

June 10, 2026

Contents

1 Introduction **2**

1.1 Literature review 3

2 Methodology **6**

2.1 Methods and technologies used 6

2.1.1 Project requirements 6

2.1.2 Technologies selection 7

2.1.3 Implementation 9

2.1.4 Object Detection Model 15

3 Results **24**

4 Conclusion **25**

5 Future work **26**

Reference list **27**

Chapter 1

Introduction

Over the past four decades, there was a major leap in robotics, especially in the field of quadrupeds. Many great papers developing the domain were published leading to significant advancements. The jump was from early days one-legged hopping machines to a four-legged "trotting" robots able to traverse through difficult terrain and obstacles, and even autonomous navigation in unstructured environments. Legged locomotion showed that it is one of the most versatile forms of mobility. But most of the research was done in labs, requiring substantial resources for prototyping and pushing the boundaries of the domain, which creates a gap between lab research and consumer-level accessibility.

The objective of this thesis is to address that gap by designing, constructing, and controlling a quadruped robot as an integrated project. The work covers the full development pipeline: mechanical design using modern CAD tools and 3D printing, selection and characterization of actuators, implementation of statically stable gaits based on established stability criteria, development of a control system, integration of computer vision, and validation through physical experiments. By following this end-to-end process, the thesis demonstrates how classical locomotion and computer vision principles can be realized on a budget-friendly platform.

A deliberate choice was made to use low-cost, commercially available components and rapid-prototyping techniques wherever possible. This decision was not driven by cost reduction alone; rather, it ensures that the resulting platform is reproducible. In this paper, I would like to review the research and practical achievements that have been done throughout the years

and combine it in a project, the cost of which will be less. The idea is to establish a base for low-cost quadruped robots to make them more affordable for day-to-day use in different fields of human lives.

1.1 Literature review

There is a number of "classical" papers that can be used to lay the foundation for this project. They vary from gaits and control to computer vision and successful real-world examples.

The most important thing for a quadruped robot is locomotion. The mathematical analysis in the 1968 "On the stability properties of quadruped creeping gaits" Mathematical Biosciences journal article shows, that there is "a unique optimum gait that maximizes static stability". The work explains that for static stability the center of gravity must be within the support polygon. This shape is formed by connecting all feet that touch the ground (contact points). Though, stability can be achieved by dynamic means as well. The concept of dynamic stability is discussed in the 1984 "Experiments in Balance with a 3D One-Legged Hopping Machine", which explains the control algorithms: forward velocity, body altitude, and hopping height. In the recent years, compute became more accessible and the development of neural networks allowed to creation of simulated environments, in which digital versions of robots were put in and given a task to arrive from point A to B without being programmed to do so using Deep Reinforcement Learning (DRL). This approach was used in 2019 "Learning Agile and Dynamic Motor Skills for Legged Robots" for ANYmal robot. The researchers applied the real-world behavior of the robot's motors to the simulation that allowed the simulation to be more realistic and helped the robot learn imperfect nuances of the world.

While the theoretical foundations establish how quadruped locomotion works, several landmark systems demonstrate what is achievable with mature engineering. Boston Dynamics' BigDog (2008) was the first to demonstrate load-carrying dynamic locomotion on rough terrain, proving quadrupeds could operate in real-world conditions outside the lab. It was a direct descendant of the 1984 hopping machine experiments and it made evident that dynamic stability is the key to traversing challenging terrain. Similarly, MIT's Cheetah 3 (2018) showed that model-based control could achieve robust blind locomotion over obstacles. These systems represented the state-of-the-art capability of quadruped robots at their time. The projects were

sponsored by DARPA and MIT respectively and did not focus on cost reduction, but rather on pushing the boundaries of what quadruped robots could do.

Another key component of the project is computer vision. Generally, robots can be controlled by a human operator, which remains a very practical approach. However, computer vision is a significant step towards autonomy and high-level task execution. Vision is critical for navigation, obstacle avoidance, and interaction with the environment. For this project, I decided to use YOLO, a state-of-the-art object detection model, based on the "You Only Look Once" (2016) architecture, which was a game-changer in the field of real-time object detection. The model divides the image into a grid, and each grid cell is responsible for predicting bounding boxes, confidence score, and class probabilities of an object whose center falls into that cell. As described in the paper: "[The] network architecture is inspired by the GoogLeNet model for image classification... [It] has 24 convolutional layers followed by 2 fully connected layers. Instead of the inception modules used by GoogLeNet,... [it simply uses] 1×1 reduction layers followed by 3×3 convolutional layers...". The original YOLO uses the Sum-Squared Error (SSE) loss function between the predictions and the ground truth. While the architecture has evolved significantly in subsequent versions to utilize CSPDarknet, a high-performance convolutional neural network backbone, and advanced loss functions, the fundamental principle of grid-based single-pass detection remains the core of the model used in this project. Additionally, it is worth mentioning Object Detection with Discriminatively Trained Part Based Models (2010), which is a classic paper on training models to recognize objects based on their parts, because quadruped robots usually have limited field of view due to their height. Applying the logic of part-based modeling to train a model to recognize an object by its lower part is crucial for the robot to "understand" the environment.

While the architecture and approach to train the model are important, the datasets used for training enable the model to reach its full potential. In this project, I utilize two of the most significant benchmarks in the history of object detection: Common Objects in Context (COCO) (2014) and Stanford Dogs (2011). The COCO dataset established the standard for detecting objects in "natural" settings rather than perfectly centered studio shots. It contains over $330,000$ images, with more than 2.5 million labeled instances across 91 object categories, including humans, dogs, cats, etc., easily recognizable by a 4-year-old. The Stanford Dogs dataset "...includes over $22,000$ annotated images of dogs belonging to 120 species. Each

image is annotated with a bounding box and object class label."

Chapter 2

Methodology

2.1 Methods and technologies used

Developing a functional quadruped robot on a limited budget requires navigating a wide landscape of hardware components, software frameworks, and design methodologies. While the robotics field offers a rich variety of actuators, computing platforms, sensors, and control strategies, the project constraints significantly narrow the range of viable options. Cost, reproducibility, and the mechanical demands of legged locomotion impose trade-offs at every stage of the design process, forcing a careful balance between performance, affordability, and ease of integration. The following subsections outline the requirements that shaped this design space and present the technologies and models selected to address them.

2.1.1 Project requirements

There are quite a lot of requirements and limitations in this project as it seeks to demonstrate a way to develop a low-cost quadruped robot. There are three big groups of limitations that define requirements. The first, and most significant, is cost. It influences the other group which is the hardware. The hardware, limited by cost, in turn constrains the software and algorithms. All together they create a really tight framework for the project, which is a good thing since it forces to creativity and finding new solutions.

My requirements for the project are the following:

- The robot should be able to walk on flat surfaces in all directions, i.e. forward, backwards, sideways, and turning in place. Flat surfaces were chosen here because rough terrain is too complex for the hardware the project uses.

- The robot should have 12 degrees of freedom (DOF), where each joint that can move independently contributes one DOF, to allow for a variety of gaits and maneuvers. Each leg should have 3 DOF: hip abduction and adduction (side-to-side movement), hip flexion and extension (forward-backward movement), and knee flexion and extension.

- It should have computer vision capabilities to detect and recognize objects. For the sake of demonstration, I chose dogs and humans as the target objects because they represent the primary agents (humans) and subjects (dogs) the robot is expected to encounter and interact with.

- The robot should be easy to assemble and reproduce by others. The components should be available, affordable, and designed with swappability in mind to allow for future upgrades and/or repairs.

On the limitation side, the cost of the whole project should not exceed 300 EUR, which is quite limiting for the result the project wants to achieve.

The hardware and software choices are affected by this limitation and the technology selection will be discussed in the next section.

2.1.2 Technologies selection

Everything starts with the model. Nowadays, 3D printers are quite accessible for people making them a great choice for the project since it allows producing custom elements perfectly fitting and satisfying the requirements and at the same time being easily reproducible by the others. At my disposal were Bambu Lab printers A1 mini, X1C Carbon and H2D series. I tried them all and settled on the last one since it has two nozzles, allowing the use of two types of filament at the same time: PLA and PETG. PLA is a great general material for the robot itself, while PETG was used for supporting elements. It is possible to use only PLA, and I have tried that, but PETG makes life easier since it does not stick to PLA, making it a perfect choice for supports. The design of the robot was done in Onshape, a free cloud-based CAD web software

with enough functionality for the project. For the slicer, which is software that converts 3D models into G-code instructions for the printer, specifying toolpaths, layer heights, and print parameters, Bambu Lab's Bambu Studio was used as it perfectly works with the printers and has the sufficient functionality for the project. I have not created the initial model by myself, as it would take much more time, but rather found a suitable one with Creative Commons (4.0 International License) on the Internet made by Tom Knox and modified it to fit the requirements. The requirements not met by the initial model were 12 DOF and ease of assembly. What the original model did provide was a solid base and design I wanted, 3 DOF per front leg, and a place for the camera module.

What made the original 3D model hard to assemble was the fact that it consisted of 149 unique parts connected by different types of screws. The most problematic part was the leg. All four legs are the same just mirrored, which made redesigning much easier. To reduce complexity of assembly, I redesigned legs and the body to be print-in-place, meaning ready to use parts without the need of additional assembly. I also reduced the number of unique screws in the project to just 1 type, which is M3 of two sizes.

For the actuators, I chose WS-MG996R digital servos with metal gears, which are widely used in hobby robotics, 3 per leg, which gives 12 DOF in total. These servos are really affordable and deliver 11 $kg \cdot cm$ stall torque with a speed of 0.17–0.14 seconds per 60°, operating at 4.8V to 7.2V DC. On the other hand, cheaper actuators mean low precision, feedback, and torque, which are necessary for this type of project. Considering all pros and cons, these are the bare minimum for the project.

For the main controller board, I chose Raspberry Pi 4 4GB. It is a nice little computer powerful enough for controlling servos and running small object recognition models with the following characteristics: a 1.5GHz 64-bit Broadcom BCM2711 Quad-core Cortex-A72 (ARM v8) processor with on-chip VideoCore VI GPU, supporting OpenGL ES 3.1 and Vulkan 1.2, Dual-band 2.4 GHz and 5.0 GHz IEEE 802.11ac wireless LAN (Wi-Fi 5) and Bluetooth 5.0 (BLE), a Gigabit Ethernet port, two USB 3.0 ports and two USB 2.0 ports, a PCIe 2.0 x1 interface, 40 pin GPIO header, two 2-lane MIPI camera/display transceivers, a USB-C power supply port, and 4GB LPDDR4-3200 SDRAM. A big advantage of this computer is that it runs Raspberry Pi OS, which is a Linux based operating system, allowing fast prototyping and running existing programs and tools. The Raspberry Pi 4 offers significant improvements over its predecessor,

including faster GPIO and better overall performance, making it well-suited for this application. The Raspberry Pi does not have enough GPIO pins to control all 12 servos, so I went with PCA9685, a 16-channel, 12-bit PWM (Pulse Width Modulation) driver chip. It uses I2C communication protocol and takes just 4 Raspberry Pi GPIO pins: VCC 3.3V, SDA, SCL, and GND. The servos consumption can spike up to 2.5A per servo, so in total it is 30A, which is way more than the Raspberry Pi can provide. To solve this problem, I decided to use an external power supply, which is 4 18650 2500 mAh Lithium-ion batteries able to produce enough current for the servos. Connected in a 2S2P configuration (two series pairs in parallel), they provide 7.4V with a total capacity of 5000 mAh. With that the potential difference is larger than what the servos and Raspberry Pi can handle, so I use buck converters to step down the voltage to 7V for the servos and 5V for the Raspberry Pi.

For the computer vision part, I use a 5-megapixel Raspberry Pi Camera Module v1.3, which features an OmniVision OV5647 Color CMOS sensor. The camera has a 3.6 mm focal length lens with a $54^\circ \times 41^\circ$ angle of view, providing a horizontal field of view roughly equal to the distance from the subject at typical operating ranges. The camera uses a CSI ribbon cable to connect to the high-speed, dual-purpose MIPI port on the Raspberry Pi, offering lower latency and CPU overhead compared to USB cameras.

On the software side, I use Python as the main programming language. It is a great choice for rapid prototyping and has a huge ecosystem of libraries for robotics, computer vision, and machine learning. For controlling the servos, I use the Adafruit PCA9685 library, which provides a simple interface for sending PWM (Pulse Width Modulation) signals to the servos. PWM varies the duty cycle of a square wave to control the average voltage delivered, with pulse widths typically ranging from 1 to 2 milliseconds corresponding to 0 to 180 degrees of servo rotation. For object detection, I use the Ultralytics YOLO library, which offers a streamlined interface for YOLO-based inference and integrates well with the Raspberry Pi platform.

2.1.3 Implementation

The implementation started by downloading the original model and printing the most important part — the leg. The leg model consisted of 7 parts, which took the printer around 3 hours. After, I assembled the leg, attached the hip to a servo and connected the servo directly to the Raspberry Pi with jumper wires. Printing it and testing that the leg works was a crucial step

that helped me to verify that the model is capable of moving and having the physical model helped me understand what needed to be changed. Later, I started redesigning the leg to be print-in-place. First of all, it is important to understand the principles of such design. The most important idea is that there should be no overhanging parts, meaning that all the parts should be supported by the print itself. To achieve that, the maximum overhang angle should not exceed 45 degrees, beyond which support material would be required. Another thing to keep in mind is the clearance between the moving parts. Through experimentation, I found that 0.65 mm provides the best balance between smooth motion and minimal play, so this value was used for most clearances in the model. The height of every print was standard, 0.2 mm, and the support parameter was set for hanging parts like curves of the legs and body "ribs". The final leg consists of 6 elements: motor extension, extension link, shoulder disk, femur, secondary link, and tibia, printed as one module. After removing it from the printer bed and detaching the supports, it is movable and ready to be installed. To get to the final design, I went through around twenty iterations of the leg and a couple of iterations of the body. Several design challenges emerged during these iterations and to achieve the requirements while using the same design, I had to break the basic principles. For example, a support blocker is used for the leg to block the support from being generated in the motor extension part, creating a bridging surface that spans a gap without support. Another example, showing the challenge of designing print-in-place models, is the knee. The knee joint employs a cylindrical pivot pin that is integrated into the tibia (lower leg), with the femur (upper leg) captured between the two halves of the tibia joint, forming a sandwich-like structure. This arrangement creates overhangs on both the top and bottom surfaces of the femur where it interfaces with the tibia, requiring support material to be generated in two places: between the upper tibia surface and the femur, and between the femur and the lower tibia surface. The overhang surface is large and cannot be recovered if drooped and melted to the other part, thus support was necessary, so unfortunately, each knee has a bit of PETG support in it. PLA filament can also be used as an alternative to PETG for support, requiring breaking it by moving the knee joint after printing. Beyond the main structural parts, the model contains eight servo holders that are printed separately. Each holder weighs 5 grams and does not require support material. The holders attach to the servos with screws, while the connection to the body uses integrated pins that slot into corresponding holes, eliminating the need for additional fasteners. In total, printing the complete model requires approximately 20.5 hours and 490 grams of filament. The

body takes 7 hours and 154 grams, each of the four legs requires 3 hours and 74 grams, and the eight servo holders together take 1.5 hours and 40 grams.

When the whole model was stable and ready, I started assembling it all together. With the experience I gained, it would take around one hour to assemble everything. Connecting PCA9685 to the Raspberry Pi was a straightforward process. The control pin connection layout is the following: VCC to 3.3V, GND to GND, SDA to GPIO 2 (SDA), and SCL to GPIO 3 (SCL). PCA9685 does not power the servos from the controller, instead there is a separate V+ and GND terminals. Those are connected to an external power supply. While testing I used a lab power supply, set to 6V and 6A, which was enough for testing purposes. In order to enable I2C communication interface on the Raspberry Pi, it needed to be enabled through the Raspberry Pi Configuration tool, which can be accessed through `sudo raspi-config` under Interface Options, I2C. After that, I2C utilities should be installed by running `sudo apt install i2c-tools`. The connectivity can be verified by running `i2cdetect -y 1`, which should show the PCA9685 at address $0x40$ (hexadecimal). The address is hardcoded to $0x40$ by the manufacturer, as the default base address for this chip. In the I2C protocol, each device requires a unique 7-bit address for the master controller to identify it. The PCA9685 uses a partially configurable address scheme where the upper bits are fixed internally (binary 1000000, corresponding to hexadecimal $0x40$), while the lower six bits can be modified via external address pins labeled A^0 through A^5. This design allows up to 62 PCA9685 devices to coexist on the same bus by changing the pin configurations, enabling expansion of PWM outputs without requiring additional communication channels. For instance, setting pin A^0 high while keeping others low changes the address from $0x40$ to $0x41$. This flexible addressing makes the PCA9685 well-suited for robotics applications requiring numerous servo motors. For this project, I am not changing the default address, so it remains at $0x40$. The other pins are 16 channels for PWM output, which are connected to the servos. Each channel consists of three pins: V+ (power), GND (ground), and OUT (signal). I grouped them by leg in this order: hip extension, knee flexion, and hip abduction, so it is easier to understand which channel controls which joint. The PCA9685 board exposes sockets for the servos, so it is convenient to connect them without soldering, however, the board itself exposes the I2C pins, while Raspberry Pi exposes its GPIO pins, so I had to solder wires to connect them together. Each servo has 180° range of motion, so before attaching servos to the robot, I set: hip to 90°, knee to 60°, and abduction to 95°, to their home position. These values are hardcoded in the code and all move-

ments are relative to them. Then, the servos are attached as following: the hip abduction servo is screwed vertically to its slot in the body and a dual-arm horn is attached to the gear, after that the hip extension and knee flexion servos are put together and secured with servo holders on the sides, then another dual-arm horn is attached to the knee flexion servo, while a round disc horn is screwed to the back side of the femur. Ultimately, the horns are connected to the hip and motor extension respectively. The same procedure is repeated for all four legs. After that, the electronics are installed inside the body and the Raspberry Pi camera module is mounted to the front camera slot. With the mechanical assembly complete, the control software can command movement.

A step consists of two phases: the swing phase, where the leg is parabolically lifted and moved forward, and the stance phase, where the foot drives backward relative to the chassis, generating the ground reaction force that pushes the robot's body forward. A gait is the periodic pattern of these foot movements. To translate desired foot positions into servo angles, the system uses a lookup table approach rather than inverse kinematics. This method was chosen because the mechanical coupling in the leg design, where hip movement affects knee angle, makes standard analytical solutions impractical. The lookup table contains pre-calibrated mappings between joint angles and foot positions, created by setting different servo angles and measuring the resulting foot positions. With the hip mounting point defined as the coordinate origin, the foot position is expressed in millimeters relative to this reference. The calibration data consists of 81 measurements spanning the feasible range of hip and knee angles. Each entry records the hip angle, knee angle, and the corresponding foot coordinates, capturing the reachable workspace of the leg. When the control system requests a target foot position (x_t, y_t), it locates the nearest calibration points in the two-dimensional position space and interpolates between them using inverse distance weighting. For each calibration point i with foot position (x_i, y_i), the Euclidean distance to the target is computed as

$$d_i = \sqrt{(x_t - x_i)^2 + (y_t - y_i)^2}. \tag{2.1}$$

The four nearest points are selected, and a weight is assigned to each according to the inverse square of its distance:

$$w_i = \frac{1}{d_i^2}. \tag{2.2}$$

These weights are normalized so that they sum to unity,

$$\hat{w}_i = \frac{w_i}{\sum_{j=1}^{4} w_j}, \tag{2.3}$$

and the interpolated joint angles are obtained as the weighted average of the corresponding calibration angles:

$$\theta_{\text{hip}} = \sum_{i=1}^{4} \hat{w}_i \, \theta_{\text{hip},i}, \qquad \theta_{\text{knee}} = \sum_{i=1}^{4} \hat{w}_i \, \theta_{\text{knee},i}. \tag{2.4}$$

In practice, the implementation includes several practical adjustments: if the target lies within 0.1 mm of a calibration point, that point is returned directly without interpolation; distances below 0.001 mm are clamped to prevent division by zero; and the final interpolated angles are rounded to the nearest integer, since the servos accept only whole-degree commands. This method assigns greater influence to closer calibration points, producing smooth angle estimates even for positions that were not explicitly measured. The approach is particularly suitable for this mechanism because the mechanical coupling between the hip and knee joints, arising from the linkage geometry of the leg, precludes a closed-form analytical solution. While inverse kinematics is the conventional approach in robotics literature, empirical mapping provides a practical alternative when mechanical complexity or manufacturing tolerances make analytical models unreliable. The abduction joints, which control side-to-side leg movement, are not included in the lookup table because their motion is kinematically decoupled from the primary leg swing mechanism. These joints operate independently and are controlled through direct angle commands. Each servo is individually calibrated to account for mechanical variations and assembly differences between left and right legs. A direction multiplier ensures that positive commands produce consistent physical movements across all legs, while center offsets compensate for variations in the home position. Three distinct gaits are implemented: a crawl gait for statically stable locomotion, where legs move sequentially in a diagonal pattern; a trot gait for faster movement, where diagonal leg pairs move simultaneously; and a turning gait, where left and right legs move in opposite directions to generate rotational torque. Servo movements are interpolated over time with configurable speed, ensuring smooth motion and preventing sudden movements that could upset the robot's balance. All movements are defined relative to a home position, which serves as the reference point for calibration and initialization. Since the lookup table was measured for the left legs, the right legs use mirrored values to maintain

symmetry in movement.

The software architecture is organized into four functional layers. The lowest layer abstracts the hardware interface, managing communication with the PWM driver and providing a unified interface for setting servo angles. Above this sits the kinematics layer, which implements the lookup table and interpolation logic to translate desired foot positions into joint angles. The gait generation layer defines the periodic leg movement patterns, swing and stance trajectories, and orchestrates their execution across multiple legs according to the selected gait pattern. As discussed in Section 1.1, statically stable gaits require that the center of gravity remains within the support polygon formed by the grounded feet; the crawl gait satisfies this criterion by moving only one leg at a time, while the trot gait achieves faster locomotion by moving diagonal pairs simultaneously, trading some stability margin for speed. The highest layer comprises the application interfaces that allow human operators to command the robot. The primary control interface is a web server running on the Raspberry Pi that exposes an HTTP-based remote control endpoint. HTTP is a request-response application protocol operating over TCP, in which the client (the operator's browser) initiates a connection and sends a request, and the server returns a response. The server listens on a dedicated port and handles two types of requests. A GET request to the root path returns the HTML control page, which contains a JavaScript component that captures keyboard events. When the operator presses a key, the script issues an asynchronous POST request carrying the movement command to a specific endpoint path. The server parses the request path to determine the desired action and, because HTTP itself is stateless, maintains an internal state machine to track whether the robot is idle, standing, or executing a gait. Upon receiving a movement command, the server transitions to the moving state and spawns a background execution thread so that the gait can run without blocking the communication interface. A timeout mechanism automatically returns the robot to a resting position if no further commands are received within a few seconds, preventing the robot from remaining in an active stance indefinitely. This design prioritizes responsiveness and simplicity, making the robot accessible to operators without requiring specialized control hardware or software installation. For development and debugging, separate command-line utilities allow individual servos to be positioned manually. These tools were essential during the calibration phase and for verifying mechanical assembly, as it enables direct control of any joint without invoking the full gait system, as well as settings servos to desired angles for collecting lookup measurements.

2.1.4 Object Detection Model

Before discussing the YOLO architecture, it is important to understand the underlying technology that makes it possible: the Convolutional Neural Network (CNN). A CNN is a class of deep neural networks specifically designed for processing grid-structured data such as images. Unlike traditional fully connected networks, where every neuron is connected to every neuron in the adjacent layer, a CNN employs convolutional layers that apply learnable filters (kernels) across local regions of the input. Each filter detects specific visual features of a given image, such as edges, textures, shapes, or increasingly abstract patterns, by computing dot products between the filter weights and the local pixel values. The output of this operation is a feature map that preserves the spatial relationships within the image. CNNs also incorporate pooling layers, which progressively reduce the spatial dimensions of the feature maps, thereby decreasing computational complexity and providing a degree of translation invariance. This hierarchical feature extraction, combined with weight sharing across spatial locations, makes CNNs exceptionally effective for computer vision tasks while requiring far fewer parameters than their fully connected counterparts.

YOLO introduces the most innovative approach to object recognition. Before, systems like Deformable Parts Model (DPM) and Regression-based Convolutional Neural Network (R-CNN) repurposed classifiers to perform detection. Classifiers for each object were used to evaluate it at various locations and scales in a test image in a sliding windows manner or on potential bounding boxes. Then the post-processing stage would refine the bounding boxes and remove duplicates. Such cumbersome pipelines were slow and difficult to optimize because of heterogeneous nature of these components. The new approach that YOLO introduces uses the whole images to train the model to predict objects, which converts object detection to a single regression problem. This method uses a single convolution network to predict multiple bounding boxes and class probabilities for these boxes simultaneously, which make it swift by simplifying the pipeline. Unlike the aforementioned mechanisms, YOLO can reason globally by examining the whole image, which gives it an advantage of learning generalized representations of objects.

The model starts by dividing the input image into $S \times S$ grid. Each grid cell is responsible for detecting bounding boxes, their confidence, and class probabilities. Grid cells are not limited to one bounding box or class, instead, they can have B boxes but only one set of C classes

probabilities per cell. In the original paper the authors suggest 2 and 20 respectively on a 7×7 grid. Each of these bounding boxes is made up of 5 numbers: the x and y positions, representing the location of the center of the predicted bounding box; the width and height relative to the entire image size; and the confidence score, representing the predicted Intersection over Union (IoU) between the bounding box and the ground truth. Formally,

$$\text{confidence} = \Pr(\text{object}) \times \text{IoU}_{\text{pred}}^{\text{truth}}. \tag{2.5}$$

If no object exists in the cell, the confidence score should ideally be zero. The class probabilities are conditioned on the cell containing an object, denoted $\Pr(\text{class}_i \mid \text{object})$. At test time, the class-specific confidence for each box is obtained by multiplying the conditional class probabilities by the box confidence:

$$\Pr(\text{class}_i \mid \text{object}) \times \Pr(\text{object}) \times \text{IoU}_{\text{pred}}^{\text{truth}} = \Pr(\text{class}_i) \times \text{IoU}_{\text{pred}}^{\text{truth}}. \tag{2.6}$$

This product encodes both the probability that the box contains an object of class i and how well the box fits the object.

The network architecture borrows heavily from GoogLeNet, also known as Inception-v1. It is made up of 24 convolutional layers followed by 2 fully connected layers at the end. The convolutional layers handle feature extraction, starting with simple edges and textures and building up to more complex patterns, while the fully connected layers map these features to the final output tensor of size $S \times S \times (B \times 5 + C)$.

For training, the authors chose Sum-Squared Error (SSE) as the loss function. This single loss covers three things at once: the accuracy of bounding box coordinates, the confidence score, and the class prediction. A challenge here was that most grid cells in any image do not contain an object, which can bias the model toward always predicting "no object." To prevent this, the loss function applies different weights to different parts of the prediction. Coordinate errors are multiplied by $\lambda_{\text{coord}} = 5$ to increase their importance, while confidence errors for empty boxes are scaled down by $\lambda_{\text{noobj}} = 0.5$. This ensures that the network pays more attention to getting the boxes right when objects are actually present.

Because every cell predicts multiple bounding boxes, the network naturally generates many

overlapping detections around the same object. To deal with this, YOLO runs non-maximum suppression (NMS) as a post-processing step. NMS works by selecting the box with the highest confidence and removing any other box that overlaps with it significantly, meaning any box with a high IoU. The end result is a single, high-confidence bounding box per object rather than multiple overlapping predictions.

Since its original release, YOLO has evolved through several major architectural shifts. Early iterations moved from pure regression to anchor-based detection, introduced batch normalization and multi-scale training, and adopted deeper residual backbones with feature pyramid-style predictions at multiple resolutions. The next wave of improvements focused on training efficiency and deployment accessibility: Cross-Stage Partial networks reduced computational cost, the ecosystem migrated from Darknet to native PyTorch, and automated augmentation and anchor calculation removed much of the manual tuning. More recently, the architecture has shifted toward anchor-free detection, decoupled classification and regression heads, and unified frameworks that integrate object detection with segmentation and pose estimation under a single model family. For this project, I decided to use the Ultralytics YOLO library with the YOLOv8 model, which implements a modern version of the architecture with these advancements built in. It offers a good trade-off between real-time performance and detection accuracy, which is critical when running on a resource-constrained platform like the Raspberry Pi. This setup allows the robot to reliably detect objects at frame rates that are sufficient for real-world interaction.

As mentioned before in the requirements of the project, the selected objects to train the model on are dogs and humans. I chose the most significant datasets in the history of object detection: COCO and Stanford Dogs. COCO contains over $330,000$ images and more than 2.5 million labeled instances across 91 object categories, while the latter contains over $22,000$ annotated images of dogs belonging to 120 species. Even though the datasets are great as they are, they should be adapted to the case I have, which is a close-to-ground quadruped robot. The average height of the camera of the robot in the standing position is 17 cm with an upward tilt of 15°.

To understand why adaptation is necessary, it helps to look at what the camera actually sees. The camera has a horizontal field of view of 54° and a vertical field of view of 41°. The optical axis is tilted 15° above horizontal, so the vertical field of view is split around this axis: the top edge of the frame is $15^\circ + 20.5^\circ = 35.5^\circ$ above horizontal, and the bottom edge is

$15^\circ - 20.5^\circ = 5.5^\circ$ below horizontal.

At a horizontal distance d from the robot, the height of the top edge of the frame is

$$y_{\text{top}} = 17 + d \cdot \tan(35.5^\circ) \approx 17 + 0.713d \text{ cm}, \tag{2.7}$$

and the height of the bottom edge is

$$y_{\text{bottom}} = 17 - d \cdot \tan(5.5^\circ) \approx 17 - 0.096d \text{ cm}. \tag{2.8}$$

The factor 0.713 comes from $\tan(35.5^\circ)$ and the factor 0.096 comes from $\tan(5.5^\circ)$. The total vertical span of the frame at distance d is the difference between these two values, which works out to approximately $0.809d$ cm. Anything outside the band $[y_{\text{bottom}}, y_{\text{top}}]$ is cut off.

The horizontal width of the view is determined by the 54° horizontal field of view. With half of that angle being 27°, the total width is

$$w = 2d \cdot \tan(27^\circ) \approx 1.019d \text{ cm}. \tag{2.9}$$

Figure 2.1 illustrates the geometry in a side view. The camera sits at 17 cm above the ground and looks upward at 15°. The vertical field of view of 41° is split into 20.5° above and below the optical axis, giving the top edge at 35.5° and the bottom edge at 5.5° below horizontal.

Figure 2.2 shows the horizontal field of view from above. The camera sees a wedge-shaped region that widens with distance.

Table 2.1 shows the resulting numbers for a few typical distances. The "Visible height range" column gives the total vertical span of the frame ($y_{\text{top}} - y_{\text{bottom}}$). The "Human visible" and "Dog visible" columns show what fraction of a standing human (170 cm tall) and a medium-sized dog (50 cm tall) actually falls inside that span. For example, at $d = 50$ cm the frame covers the band from 12.2 cm to 52.7 cm above the ground. A human standing there occupies $[0, 170]$ cm, so the overlap is $[12.2, 52.7]$ cm, giving a visible portion of 40.5 cm or about 24% of the total body. A dog of height 50 cm occupies $[0, 50]$ cm, so the overlap is $[12.2, 50]$ cm, giving 37.8 cm or about 76% of the body.

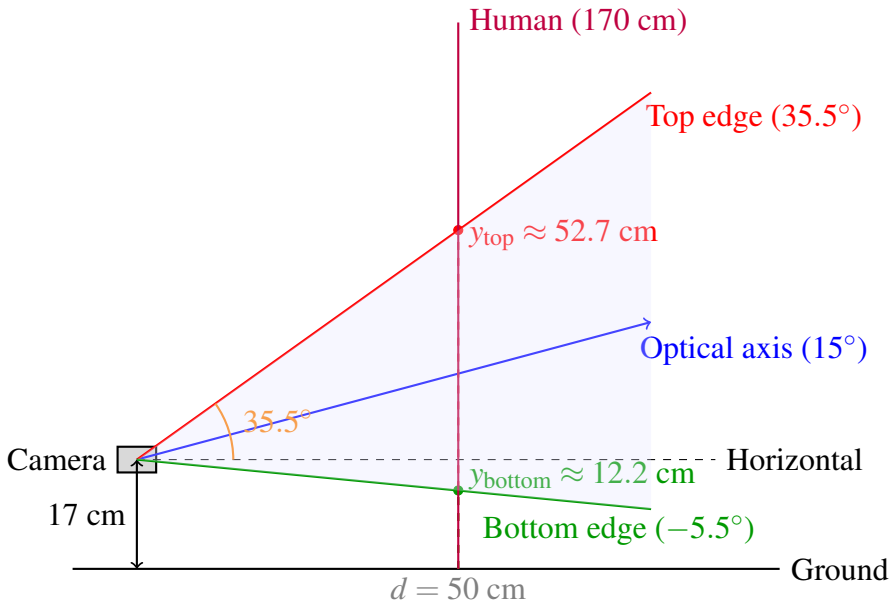


Figure 2.1: Side view of the camera geometry showing the vertical field of view at a distance of $d = 50$ cm. The shaded region represents the camera's view frustum.

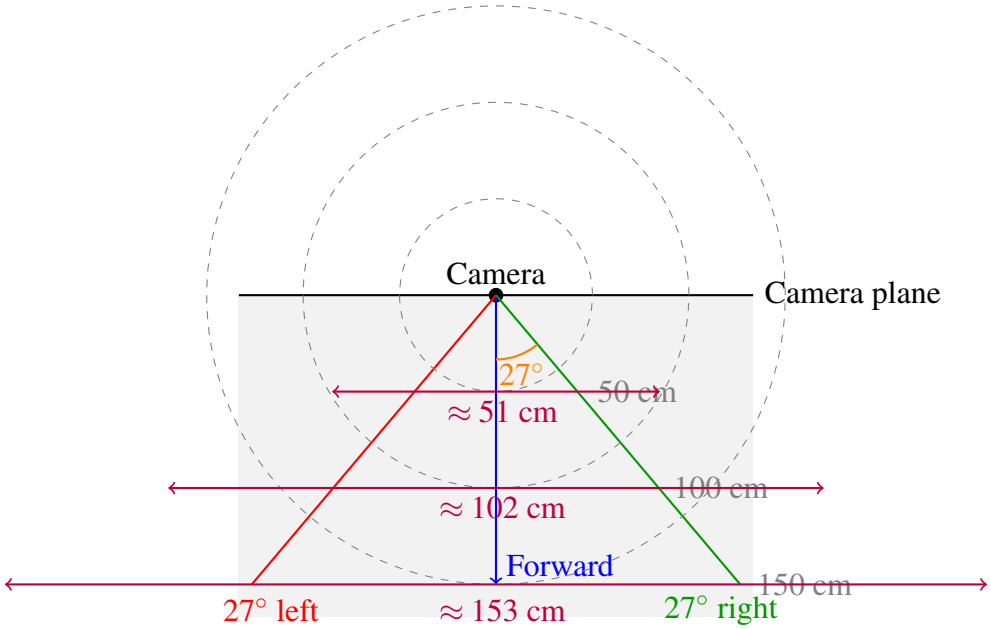


Figure 2.2: Top-down view of the horizontal field of view. The dashed circles mark distances of 50 cm, 100 cm, and 150 cm, with the corresponding view widths shown in purple.

Distance	Visible height range	Visible width	Human visible	Dog visible
50	40.5	51.0	≈24%	≈76%
100	81.0	101.9	≈48%	≈85%
150	121.4	152.9	≈71%	≈95%

Table 2.1: Camera field of view at different distances (in cm), assuming an adult human height of 170 cm and a medium dog height of 50 cm.

The numbers make the problem clear. At the distances where the robot is likely to encounter humans and dogs, roughly 50–150 cm, the camera captures the lower portion of humans and most of a dog's body. At 50 cm, only about a quarter of a human is visible, mostly lower legs and feet. Dogs fare much better because of their smaller size, with three-quarters or more of the body typically in frame. As the distance increases, more of a human comes into view, but the perspective remains unusual: the camera looks up from below rather than straight on. This is a very different viewpoint from the standard dataset images, which are usually taken at eye level and show complete objects. Training on those images without adjustment would leave the model unprepared for the low-angle, from-below perspective that the robot actually experiences.

To address the issue, I create a balanced dataset by combining the original COCO and Stanford Dogs datasets that undergo data transformations to simulate the robot's perspective as well as to mimic variations in camera exposure, color calibration, and image quality. The balanced dataset contains cropped images of people and dogs as well as their bounding box annotations. Additionally to that, the dataset contains 10% of negative sample images, which are images that do not contain any of the target objects. After preparing the dataset, I split it into training and validation sets, with an 80–20 split, which results into $12,500$ training images, where $5,000$ contain people, $5,000$ contain dogs, and $2,500$ are negative samples. The validation set contains $2,500$ images: $1,000$ with people, $1,000$ with dogs, and 500 negative samples.

The model is trained on a MacBook Pro with an Apple M2 Pro chip with 16 core GPU. The training utilzes the Metal Performance Shaders (MPS) backend, which is Apple's framework, designed on top of Metal graphics API, for accelerating machine learning workloads on GPU. Architecturally, MPS abstracts the complexity of writing low-level Metal Shading Language (MSL) by providing a suite of pre-compiled, mathematically optimized compute kernels, which are small parallelized mathematical operations, such as multi-dimensional matrix multiplications and convolutional operations. The execution of these kernels is orchestrated by PyTorch's

dynamic computational graph, which maps out the sequence of mathematical operations and data dependencies required by the model. When this graph is dispatched to the "mps" device backend, the framework analyzes the graph's structure to select and schedule the precise kernels needed for execution. This process leverages Apple Silicon's Unified Memory Architecture (UMA), executing a zero-copy memory allocation scheme where the CPU and GPU access a singular, shared pool of physical RAM without the latency-inducing overhead of PCIe bus transfers. Consequently, MPS compiles these graph operations into specialized Pipeline State Objects (PSOs) and commits them to a Metal Command Buffer, which the hardware schedules across highly parallelized GPU compute cores. This tight integration Apple's hardware in PyTorch allows to use the full potential of the hardware, which results in significantly faster training times compared to CPU-only execution.

An additional factor that contributes to the speed of training is the correct use of training parameters. After many experiments, I found that the best resulting parameters in terms of accuracy and training time are: batch size of 8, epochs of 30, 0 workers, 0 mosaic and disable validation during training. Additionally, the image size parameter is set to 320 so the model can run in real-time on the Raspberry Pi. The final command for training is:

```
PYTORCH_MPS_HIGH_WATERMARK_RATIO=0.0 \
yolo detect train model=yolov8n.pt \
data=data.yaml \
imgsz=320 \
batch=8 \
epochs=30 \
device=mps \
workers=0 \
mosaic=0 \
val=False
```

where:

`PYTORCH_MPS_HIGH_WATERMARK_RATIO=0.0` An environment variable for PyTorch's MPS backend; setting it to `0.0` allows the MPS allocator to use memory more aggressively to avoid OOM errors on Apple Silicon.

`yolo detect train` The Ultralytics CLI command that selects the YOLO detection task and starts a training run.

`model=yolov8n.pt` The initial weights to load (YOLOv8 Nano). This uses a small, fast pre-trained model as a transfer-learning starting point.

`data=data.yaml` Path to the dataset configuration YAML describing training/validation image locations and class names.

`imgsz=320` Input image size (images are resized to 320×320). Smaller sizes reduce memory and speed up training at the cost of some small-object accuracy, acceptable for real-time applications.

`batch=8` Number of images processed per optimization step. Lower batch sizes reduce memory usage and increases training speed.

`epochs=30` Number of full passes over the training set.

`device=mps` Use the Apple Metal Performance Shaders (MPS) GPU backend on macOS instead of the CPU.

`workers=0` Disable background data loader subprocesses (safer on macOS to avoid multiprocessing issues).

`mosaic=0` Turn off Mosaic augmentation (disables the 4-image mosaic composite to reduce data-loading overhead).

`val=False` Skip running validation between epochs to speed up the training loop; validation can be run after training completes.

This configuration is well-optimized for training on consumer hardware while still achieving good accuracy. The training process typically take around 2.5 hours to complete on this hardware. The recorded training curves show steady convergence: all training losses (bounding-box, class, and distribution focal loss) decrease consistently, which indicates the model learned from the data. A final validation pass (performed after training because validation was disabled during the loop) produced very high aggregate metrics: precision $\approx 96.5\%$, recall $\approx 94.5\%$, mAP@50 $\approx 98.7\%$, and mAP@50:95 $\approx 97.8\%$. At the same time the validation classifica-

tion loss is noticeably larger than the training classification loss (val ≈ 0.70 vs. train ≈ 0.09), suggesting some degree of overfitting on class predictions. These results are encouraging for a lightweight ‘yolov8n‘ model running at 320×320.



Figure 2.3: Validation predictions on a batch with bounding box, class, and confidence scores.

Chapter 3

Results

Chapter 4

Conclusion

Chapter 5

Future work

Bibliography

[1] Raibert, M., Blankespoor, K., Nelson, G., & Playter, R. (2008). BigDog, the rough-terrain quadruped robot. *Proceedings of the 17th IFAC World Congress*, 10822–10825.

[2] Katz, B., Di Carlo, J., & Kim, S. (2019). MIT Cheetah 3: Design and control of a robust, dynamic quadruped robot. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2245–2251.

[3] McGhee, R. B., & Frank, A. A. (1968). On the stability properties of quadruped creeping gaits. *Mathematical Biosciences*, 3(3–4), 331–351.

[4] Raibert, M. H. (1984). Experiments in balance with a 3D one-legged hopping machine. *The International Journal of Robotics Research*, 3(2), 75–92.

[5] Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, D., Tsounis, V., Koltun, V., & Hutter, M. (2019). Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26), eaau5872.

[6] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788.

[7] Felzenszwalb, P. F., Girshick, R. B., McAllester, D., & Ramanan, D. (2010). Object detection with discriminatively trained part-based models. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(9), 1627–1645.

[8] Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. *Proceedings of the*

European Conference on Computer Vision (ECCV), 740–755.

[9] Khosla, A., Jayadevaprakash, N., Yao, B., & Fei-Fei, L. (2011). Novel dataset for fine-grained image categorization. *First Workshop on Fine-Grained Visual Categorization, IEEE Conference on Computer Vision and Pattern Recognition*.

[10] Knox, T. (n.d.). Quadruped I. Retrieved from `https://www.printables.com/model/1443108-quadruped-i`